Universidade de Brasília

Departamento de Ciência da Computação

Relatório do Trabalho 1 de Programação Concorrente

Evelyn Soares Pereira 17/0102785

Brasília
9 de novembro de 2026

1 Introdução

Em programação concorrente, *threads* que cooperam por memória compartilhada correm o risco de *condições de corrida*: se duas delas acessam o mesmo dado sem coordenação e pelo menos uma escreve, o resultado passa a depender da ordem em que o escalonador intercala suas instruções. Como essa ordem muda a cada execução, os erros aparecem em uma execução e podem sumir na seguinte. Para evitá-los, as *threads* são sincronizadas com mecanismos como *locks*, variáveis de condição e semáforos.

Este trabalho aplica esses mecanismos a um problema de Manna e Pnueli, apresentado por Ben-Ari (2006, Apêndice C, exercício 8): calcular o coeficiente binomial $\binom{n}{k}$ com dois processos, um para o numerador e outro para o denominador. O denominador entrega resultados parciais ao numerador, que divide assim que os recebe, impedindo que os valores intermediários cresçam demais. Essa comunicação acontece por memória compartilhada e, sem sincronização, está sujeita a condições de corrida. A solução foi implementada em C com POSIX Pthreads. A Seção 2 formaliza o problema, a Seção 3 descreve o algoritmo, a implementação e os testes, e a Seção 4 conclui o trabalho.

2 Formalização do Problema Proposto

2.1 O Coeficiente Binomial

Para inteiros $0 \le k \le n$,

$$\binom{n}{k} = \frac{n!}{(n-k)!\,k!} = \frac{n \cdot (n-1) \cdots (n-k+1)}{1 \cdot 2 \cdots k}, \tag{1}$$

ou seja, há k fatores no numerador e k no denominador. Calcular o numerador inteiro e só dividir no final é inviável mesmo para valores modestos: em $\binom{30}{15}$, o numerador $30 \cdot 29 \cdots 16 \approx 2{,}0 \times 10^{20}$ não cabe em 64 bits ($\approx 1{,}8 \times 10^{19}$), embora o resultado ($155\,117\,520$) caiba em 32. A proposta do exercício é intercalar as operações: no passo i, multiplicar o valor acumulado pelo i-ésimo fator do numerador e dividir pelo i-ésimo fator do denominador (Tabela 1).

Tabela 1: Cálculo de $\binom{10}{3}$ intercalando multiplicações e divisões.

Passo i	Multiplica por	Antes da divisão	Divide por	Depois da divisão
1	10	10	1	10
2	9	90	2	45
3	8	360	3	120

Como a divisão inteira em C descarta o resto sem aviso, esse método só funciona se toda divisão intermediária for exata, o que é provado a seguir.

2.2 Divisibilidade dos Resultados Parciais

Seja $P(i, j) = j \cdot (j+1) \cdots (j+i-1)$ o produto de i inteiros consecutivos começando em j.

Lema 1. *Para todos os inteiros $i \geq 1$ e $j \geq 1$, $i!$ divide $P(i, j)$.*

Demonstração. □

Após i multiplicações, o numerador acumulou $N_i = n(n-1)\cdots(n-i+1) = P(i,\, n-i+1)$, com $n-i+1 \geq 1$ porque $i \leq k \leq n$; pelo lema, $i!$ divide N_i (como no enunciado: $1 \cdot 2$ divide $10 \cdot 9$ e $1 \cdot 2 \cdot 3$ divide $10 \cdot 9 \cdot 8$). Se o valor acumulado já foi dividido por $1, 2, \ldots, i-1$, antes da i-ésima divisão ele vale $r_i = N_i/(i-1)!$, e $r_i/i = N_i/i! = \binom{n}{i}$ é inteiro. Logo, o denominador só precisa entregar o próximo fator i (dividir por $1, \ldots, i$ em sequência equivale a dividir por $i!$), e o maior valor intermediário, $r_i = i\binom{n}{i}$, fica muito abaixo de N_k: em $\binom{30}{15}$, cerca de $2{,}3 \times 10^9$ contra $2{,}0 \times 10^{20}$.

2.3 Processos, Dados Compartilhados e Condições de Corrida

Além da *thread* principal, que lê n e k, cria as outras duas, espera seu término e imprime o resultado, há duas *threads* numa relação produtor/consumidor:

- **Denominador** (T_d, produtor): entrega, em ordem, os fatores $1, 2, \ldots, k$;
- **Numerador** (T_n, consumidor): para $i = 1, \ldots, k$, multiplica o acumulado por $n - i + 1$, obtém o i-ésimo fator de T_d e divide por ele.

O acumulado é local a T_n (a principal só o lê após o `pthread_join`). O dado compartilhado é o *canal* entre T_d e T_n: as posições onde os fatores são depositados e as variáveis que controlam quantos estão disponíveis. Sem sincronização, há três condições de corrida: (1) **leitura antes da escrita**, em que T_n lê um valor antigo ou o inicial (se for 0, divide por zero); (2) **sobrescrita**, em que T_d deposita um fator antes de o anterior ser consumido e um divisor se perde; e (3) **atualização perdida**, porque incrementos e decrementos concorrentes de uma variável de controle (`qtd++`, `qtd--`) não são atômicos e um deles pode se perder. A intercalação abaixo, para $\binom{5}{2} = 10$, combina (1) e (2) e termina com 4, sem nenhum erro aparente:

```
T_n: acc = 1 * 5 = 5
T_d: escreve 1; escreve 2       (o 1 se perde)
T_n: lê 2 -> acc = 5 / 2 = 2    (divisão truncada)
T_n: acc = 2 * 4 = 8; lê 2 de novo -> acc = 8 / 2 = 4
```

Uma solução correta deve garantir: **entrega exata** (cada fator é consumido exatamente uma vez, na ordem em que foi produzido); **exclusão mútua** no acesso às variáveis do canal; e **término sem *deadlock*** (as duas *threads* terminam após k iterações). Com as duas primeiras, T_n executa a mesma sequência da Tabela 1 e, pelo lema, obtém $\binom{n}{k}$. É desejável, ainda, que as esperas sejam por bloqueio, e não por espera ocupada.

3 Descrição do Algoritmo Desenvolvido para Solução do Problema Proposto

3.1 Visão Geral

3.2 Mecanismos de Sincronização Utilizados

3.3 Pseudocódigo

3.4 Corretude

3.5 Implementação

3.6 Testes e Resultados

4 Conclusão

5 Referências

BEN-ARI, Mordechai. **Principles of Concurrent and Distributed Programming**. 2. ed. Harlow: Addison-Wesley, 2006. ISBN 978-0-321-31283-9.